New parent page: SonarQube Server-hosted

Install and configure the SonarQube MCP Server extension on SonarQube Server. {: shortdesc}

Overview

The SonarQube MCP Server can be installed as an extension on SonarQube Server. Once installed, SonarQube Server acts as a proxy and exposes the MCP server's tools at `<YourSonarQubeURL>/mcp`. Your AI agent connects to that single endpoint. There is no separate MCP server URL to manage.

The SonarQube Server MCP extension is available on all commercial editions: Developer, Enterprise, and Data Center Edition. {: note}

Before you start

Sonar Vortex's agentic analysis and context augmentation features require file system access and don't work with the SonarQube Server-hosted MCP server. To set them up, the recommended methods are the SonarQube plugin or SonarQube CLI. See the *Make your agent verify its code* and *Add context to generate better code* pages.

Prerequisites

- SonarQube Server 2026.3 or later
- A SonarQube Server user account with permission to generate a user token

Compatibility

SonarQube Server version	Compatible MCP Server version
2026.4	1.22.0.3040
2026.3	1.18.1.2664

Always set `MCP_VERSION` to the full 4-part version from this table (for example, use `1.18.1.2664` with SonarQube Server 2026.3). Only full 4-part tags are published to Docker Hub; 3-part tags like `1.18.1` don't exist.

Choose your installation method

Your setup	Installation guide
Run SonarQube Server from a downloaded ZIP/JAR and manage the process yourself (including as a systemd or Windows service)	Install from ZIP
Run SonarQube Server in Docker or Docker Compose, on any edition including Data Center	Install from Docker
Run SonarQube Server on Kubernetes or OpenShift, via Helm	Install on Kubernetes / OpenShift

Configuration reference

These properties configure the MCP proxy on SonarQube Server, regardless of which method you used to install it. Set them in conf/sonar.properties, or as environment variables on the SonarQube container.

Property (sonar.properties / ENV variable)	Description
sonar.mcp.enabled / SONAR_MCP_ENABLED	Enables or disables the MCP proxy. (default: true)
sonar.mcp.serverUrl / SONAR_MCP_SERVERURL	The URL where the MCP server will be running
sonar.mcp.healthCheckInterval / SONAR_MCP_HEALTHCHECKINTERVAL	Health check interval. (default: 30s, optional)

Connect your AI agent

Once the MCP server extension is running, configure your AI agent to connect to it.

Generate a user token : In SonarQube Server, go to My Account > Security and generate a user token.

Configure your agent : Use the official SonarQube MCP Server configuration generator to get a configuration snippet for your setup:

- Identify your target MCP client.
- Find your environment variables.
  - Select SonarQube Server as your instance.
  - Use environment variable substitution to pass your token, or enter your SonarQube Server user token directly in the User token (optional) field.
- Choose a hosting method:
  - Select Remote server (HTTP).
  - Fill the full server URL with `https://<YourSonarQubeURL>/mcp`. Don't add a trailing slash.
- Copy the configuration and paste it into your terminal.
  - Optional: To make the MCP server available globally and not only for the repo where the command has been run, append --scope user to your CLI command.
  - Optional: Verify that the SonarQube entry has been added to your agent configuration file with your user token and the new URL.
- Restart your agent after saving the configuration and run /mcp to verify it's working.

Check the status

After installation, verify the extension is running correctly. In SonarQube Server, go to **Administration** > **System info** > **System** > **MCP** and check the **Enabled** and **Healthy** statuses.

New child page: Install from Docker

Run the SonarQube MCP Server as a Docker container alongside SonarQube Server, using Docker Compose or as a standalone container next to a ZIP-based install. {: shortdesc}

Docker-specific prerequisites

- Docker / Docker Compose
- Check the MCP server version compatible with your SonarQube Server version

The examples below use Docker Compose. You can replace any of them with equivalent plain docker run commands if you prefer. {: note}

Option 1: Full Docker Compose (developer and enterprise edition)

Add an mcp service alongside your sonarqube service and pass the MCP connection properties as environment variables on the SonarQube Server container. The example below uses the developer edition. Replace the tag developer with enterprise to deploy the Enterprise edition instead.

See an example on GitHub.

```
services:
  sonarqube:
    image: sonarqube:${SONARQUBE_VERSION}
    hostname: sonarqube
    container_name: sonarqube
    read_only: true
    depends_on:
      db:
        condition: service_healthy
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://db:5432/sonar
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar
      SONAR_MCP_ENABLED: "true"
      SONAR_MCP_SERVERURL: "http://mcp:8080"
      SONAR_MCP_HEALTHCHECKINTERVAL: "30"
    volumes:
      - sonarqube_data:/opt/sonarqube/data
      - sonarqube_extensions:/opt/sonarqube/extensions
      - sonarqube_logs:/opt/sonarqube/logs
      - sonarqube_temp:/opt/sonarqube/temp
    tmpfs:
      - /tmp:size=256M,mode=1777
    healthcheck:
      test: ["CMD-SHELL", "curl -sf http://localhost:9000/api/system/status | grep
-q '\"status\":\"UP\"'"]
      interval: 30s
      timeout: 10s
      retries: 10
      start_period: 120s
    ports:
      - "9000:9000"
    networks:
      - sonarnet

  mcp:
```

```
    image: sonarsource/sonarqube-mcp:${MCP_VERSION}
    hostname: mcp
    container_name: sonarqube-mcp
    depends_on:
      sonarqube:
        condition: service_healthy
    environment:
      STORAGE_PATH: /data
      SONARQUBE_URL: http://sonarqube:9000
      SONARQUBE_HTTP_HOST: mcp
      SONARQUBE_HTTP_PORT: 8080
      SONARQUBE_TRANSPORT: http
    volumes:
      - mcp_data:/data
    ports:
      - "8080:8080"
    networks:
      - sonarnet

  db:
    image: postgres:18
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -d $${POSTGRES_DB} -U $${POSTGRES_USER}"]
      interval: 10s
      timeout: 5s
      retries: 5
    hostname: postgresql
    container_name: postgresql
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar
      POSTGRES_DB: sonar
    volumes:
      - postgresql:/var/lib/postgresql
    networks:
      - sonarnet

volumes:
  sonarqube_data:
  sonarqube_temp:
  sonarqube_extensions:
  sonarqube_logs:
  postgresql:
  mcp_data:

networks:
  sonarnet:
    driver: bridge
```

For SonarQube MCP environment variables, see the configuration reference section.

MCP container environment variables for Option 1

Variable	Example	Description
STORAGE_PATH	/data	Writable directory for MCP logs, plugins, and temp files
SONARQUBE_URL	Sonar Qube	Example URL of the SonarQube Server instance that the MCP server connects to
SONARQUBE_HTTP_PORT	8080	Port the MCP server listens on
SONARQUBE_TRANSPORT	http	Transport mode

See also environment variables.

Option 2: Docker Compose (Data Center edition)

The Data Center Compose file follows the same pattern, scaled across multiple SonarQube and search nodes. See the full example on GitHub.

```
services:
  sonarqube:
    deploy:
      replicas: 2
    healthcheck:
      test: curl -s http://localhost:9000/api/system/status | grep -q -e
'"status":"UP"' -e '"status":"DB_MIGRATION_NEEDED"' -e
'"status":"DB_MIGRATION_RUNNING"'
      interval: 25s
      timeout: 1s
      retries: 3
      start_period: 240s
    image: sonarqube:${SONARQUBE_VERSION}
    read_only: true
    depends_on:
      search-1:
        condition: service_healthy
      search-2:
        condition: service_healthy
      db:
        condition: service_healthy
    networks:
      - ${NETWORK_TYPE:-ipv4}
    cpus: 0.5
    mem_limit: 4096M
    mem_reservation: 4096M
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://db:5432/sonar
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar
      SONAR_WEB_PORT: 9000
      SONAR_CLUSTER_SEARCH_HOSTS: "search-1,search-2,search-3"
      SONAR_CLUSTER_HOSTS: "sonarqube"
      SONAR_CLUSTER_KUBERNETES: true
      SONAR_AUTH_JWTBASE64HS256SECRET:
```

```
"dZ0EB0KxnF++nr5+4vfTCaun/eWbv6gOoXodiAMqcFo="
      SONAR_MCP_ENABLED: "true"
      SONAR_MCP_SERVERURL: "http://mcp:8080"
      SONAR_MCP_HEALTHCHECKINTERVAL: "30"
    volumes:
      - sonarqube_extensions:/opt/sonarqube/extensions
      - sonarqube_logs:/opt/sonarqube/logs
      - sonarqube_temp:/opt/sonarqube/temp
      - /opt/sonarqube/data

  search-1:
    image: sonarqube:${SONARQUBE_VERSION}
    read_only: true
    hostname: "search-1"
    cpus: 0.5
    mem_limit: 3072M
    mem_reservation: 3072M
    depends_on:
      db:
        condition: service_healthy
    networks:
      - ${NETWORK_TYPE:-ipv4}
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://db:5432/sonar
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar
      SONAR_CLUSTER_ES_HOSTS: "search-1,search-2,search-3"
      SONAR_CLUSTER_NODE_NAME: "search-1"
    volumes:
      - search_data-1:/opt/sonarqube/data
      - sonarqube_logs:/opt/sonarqube/logs
      - search_temp-1:/opt/sonarqube/temp
      - search_logs-1:/opt/sonarqube/logs
    tmpfs:
      - /tmp:size=256M,mode=1777
    healthcheck:
      test: curl -s "http://$$SONAR_CLUSTER_NODE_NAME:9001/_cluster/health?
wait_for_status=yellow&timeout=50s" | grep -q -e '"status":"green"' -e
'"status":"yellow"'; if [ $? -eq 0 ]; then exit 0; else exit 1; fi
      interval: 25s
      timeout: 1s
      retries: 3
      start_period: 55s

  search-2:
    image: sonarqube:${SONARQUBE_VERSION}
    read_only: true
    hostname: "search-2"
    cpus: 0.5
    mem_limit: 3072M
    mem_reservation: 3072M
    depends_on:
      db:
        condition: service_healthy
```

```
    networks:
      - ${NETWORK_TYPE:-ipv4}
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://db:5432/sonar
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar
      SONAR_CLUSTER_ES_HOSTS: "search-1,search-2,search-3"
      SONAR_CLUSTER_NODE_NAME: "search-2"
    volumes:
      - search_data-2:/opt/sonarqube/data
      - sonarqube_logs:/opt/sonarqube/logs
      - search_temp-2:/opt/sonarqube/temp
      - search_logs-2:/opt/sonarqube/logs
    tmpfs:
      - /tmp:size=256M,mode=1777
    healthcheck:
      test: curl -s "http://$$SONAR_CLUSTER_NODE_NAME:9001/_cluster/health?
wait_for_status=yellow&timeout=50s" | grep -q -e '"status":"green"' -e
'"status":"yellow"'; if [ $? -eq 0 ]; then exit 0; else exit 1; fi
      interval: 25s
      timeout: 1s
      retries: 3
      start_period: 55s

  search-3:
    image: sonarqube:${SONARQUBE_VERSION}
    read_only: true
    hostname: "search-3"
    cpus: 0.5
    mem_limit: 3072M
    mem_reservation: 3072M
    depends_on:
      db:
        condition: service_healthy
    networks:
      - ${NETWORK_TYPE:-ipv4}
    environment:
      SONAR_JDBC_URL: jdbc:postgresql://db:5432/sonar
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar
      SONAR_CLUSTER_ES_HOSTS: "search-1,search-2,search-3"
      SONAR_CLUSTER_NODE_NAME: "search-3"
    volumes:
      - search_data-3:/opt/sonarqube/data
      - sonarqube_logs:/opt/sonarqube/logs
      - search_temp-3:/opt/sonarqube/temp
      - search_logs-3:/opt/sonarqube/logs
    tmpfs:
      - /tmp:size=256M,mode=1777
    healthcheck:
      test: curl -s "http://$$SONAR_CLUSTER_NODE_NAME:9001/_cluster/health?
wait_for_status=yellow&timeout=50s" | grep -q -e '"status":"green"' -e
'"status":"yellow"'; if [ $? -eq 0 ]; then exit 0; else exit 1; fi
      interval: 25s
```

```
      timeout: 1s
      retries: 3
      start_period: 55s

  db:
    image: postgres:18
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -d $${POSTGRES_DB} -U $${POSTGRES_USER}"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - ${NETWORK_TYPE:-ipv4}
    environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar
      POSTGRES_DB: sonar
    volumes:
      - postgresql:/var/lib/postgresql

  mcp:
    image: sonarsource/sonarqube-mcp:${MCP_VERSION}
    hostname: mcp
    container_name: sonarqube-mcp
    depends_on:
      sonarqube:
        condition: service_healthy
    environment:
      STORAGE_PATH: /data
      SONARQUBE_URL: http://sonarqube:9000
      SONARQUBE_HTTP_HOST: mcp
      SONARQUBE_HTTP_PORT: 8080
      SONARQUBE_TRANSPORT: http
    volumes:
      - mcp_data:/data
    ports:
      - "8080:8080"
    networks:
      - ${NETWORK_TYPE:-ipv4}

networks:
  ipv4:
    driver: bridge
    enable_ipv6: false
  dual:
    driver: bridge
   See the full list of `mcp.*` values in the [SonarQube Helm chart README]
(https://github.com/SonarSource/helm-chart-
sonarqube/blob/master/charts/sonarqube/README.md#mcp-model-context-protocol).
    ipam:
      config:
        - subnet: "192.168.3.0/24"
          gateway: "192.168.3.1"
        - subnet: "2001:db8:3::/64"
```

```
            gateway: "2001:db8:3::1"

  volumes:
    sonarqube_extensions:
    sonarqube_logs:
    sonarqube_temp:
    search_data-1:
    search_data-2:
    search_data-3:
    search_temp-1:
    search_temp-2:
    search_temp-3:
    search_logs-1:
    search_logs-2:
    search_logs-3:
    postgresql:
    mcp_data:
```

For the full list of mcp.* values available in this setup, see the SonarQube Helm chart README.

MCP container environment variables for Option 2

Variable	Example	Description
STORAGE_PATH	/data	Writable directory for MCP logs, plugins, and temp files
SONARQUBE_URL	Sonar Qube	Example URL of the SonarQube Server instance that the MCP server connects to
SONARQUBE_HTTP_PORT	8080	Port the MCP server listens on
SONARQUBE_TRANSPORT	http	Transport mode

Option 3: Hybrid (SonarQube Server from ZIP + MCP as Docker container)

Use this option when SonarQube Server is installed from a ZIP archive and you want to run the MCP server as a Docker container.

Step 1 : Configure SonarQube Server

In `conf/sonar.properties`, set:

```
sonar.mcp.enabled=true
sonar.mcp.serverUrl=http://<YourMCPServerHostname>:8080
# Optional: sonar.mcp.healthCheckInterval=30
```

Then restart SonarQube Server for the changes to take effect.

Step 2 : Start the MCP container

```
docker run -d \
  --name sonarqube-mcp \
  -e STORAGE_PATH=/data \
  -e SONARQUBE_URL=http://<YourSonarQubeHostname>:9000 \
  -e SONARQUBE_HTTP_HOST=0.0.0.0 \
  -e SONARQUBE_HTTP_PORT=8080 \
  -e SONARQUBE_TRANSPORT=http \
  -v mcp_data:/data \
  -p 8080:8080 \
  sonarsource/sonarqube-mcp:${MCP_VERSION}
```

Replace `<YourSonarQubeHostname>` with the hostname or IP address reachable from within the container.

Next step

Once your MCP extension is running, go back to the parent page to configure your AI agent and check the status.